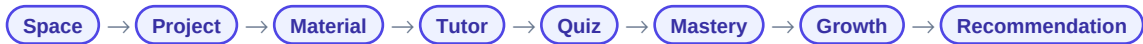


StudyFlowAI

AI Study Companion · Architecture Documentation

StudyFlowAI is a persistent learning companion: upload study material, ask a grounded AI tutor, take adaptive quizzes, and track mastery, growth and recommendations — with every AI call observable. Built with Express 5 + MongoDB + React (MERN) and Google Gemini.

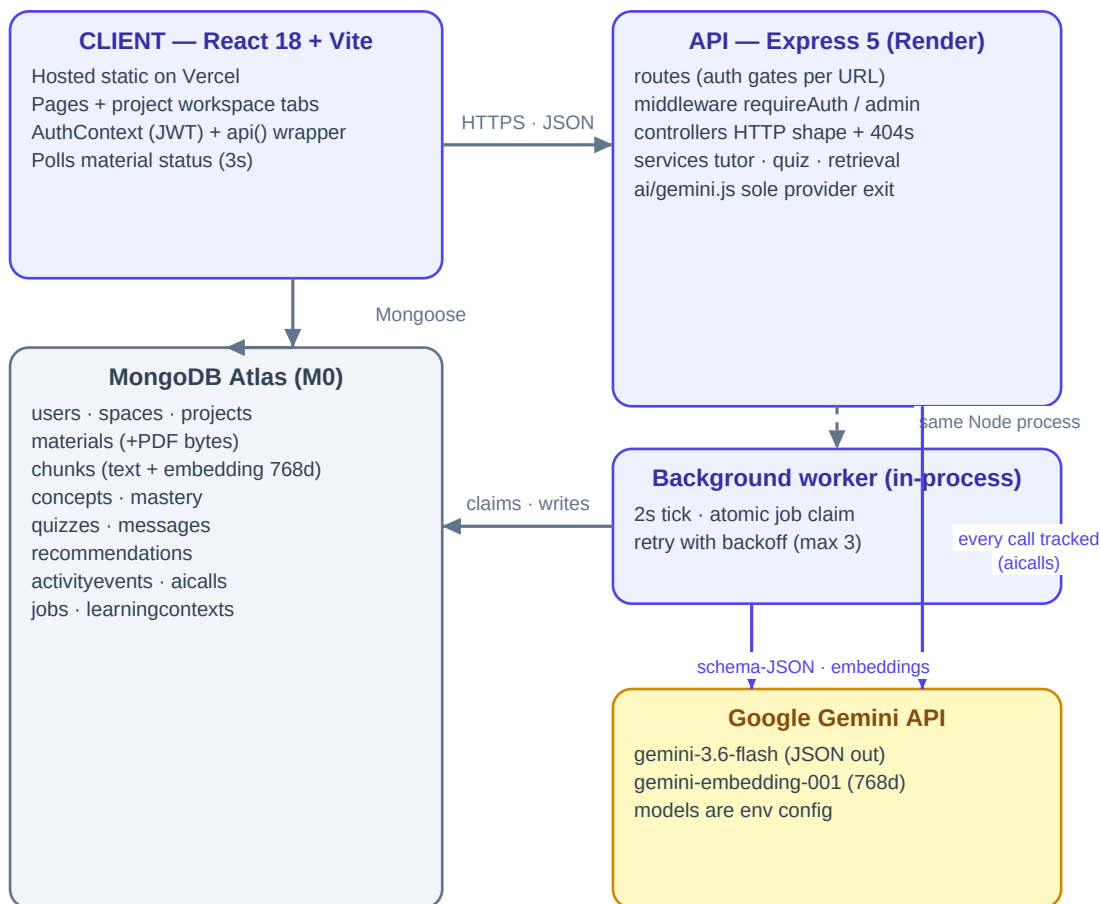
The core loop



1 · Stack at a glance

Layer	Choice	Why
API	Node 20 · Express 5 · Mongoose 8	One language end-to-end; async handlers flow into a central error handler
Database	MongoDB Atlas (M0)	Documents match AI-shaped aggregates; free tier; vector search upgrade path
AI	Gemini via @google/genai	Schema-constrained JSON + task-typed embeddings from one gateway
Frontend	React 18 · Vite · Tailwind 4	SPA with per-project workspace tabs; fast builds
Hosting	Render (API) + Vercel (web) + Atlas	Managed free tiers, auto-deploy from GitHub
Tests	node --test + mongodb-memory-server	Offline suite (15 tests / 6 suites) with AI stubbed at seams

2 · System architecture



One learning loop per project: upload → grounded tutor → adaptive quiz → mastery blend → recommendation — every query is data-isolated by owner/project.

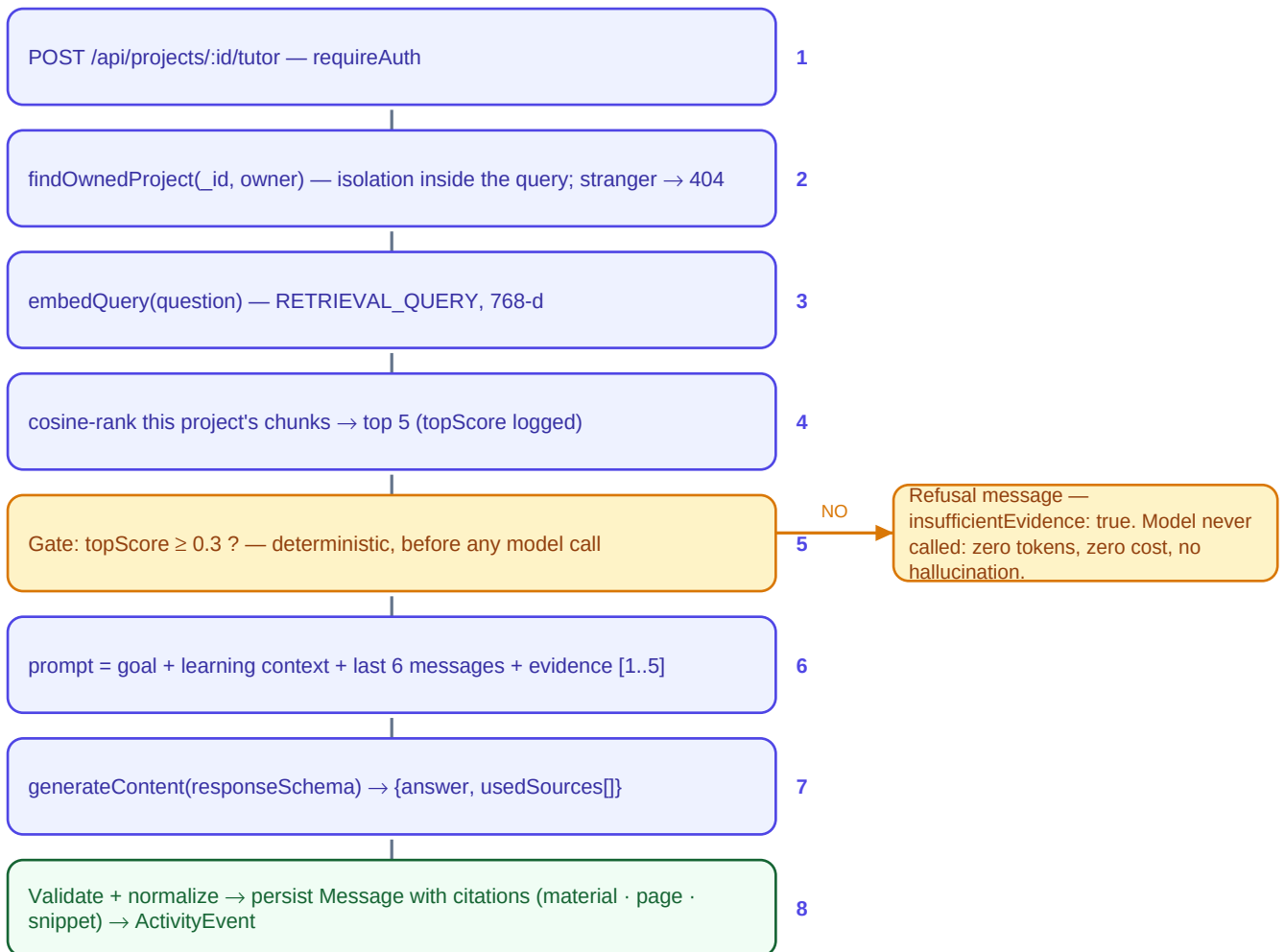
Layering. routes/ (URL wiring, auth gates) → controllers/ (validation, ownership, HTTP shape) → services/ (business logic) → ai/gemini.js (the only module that talks to the provider) → models/ (Mongoose schemas). Cross-cutting concerns live in middleware/ and workers/.

3 · Data model (13 collections)

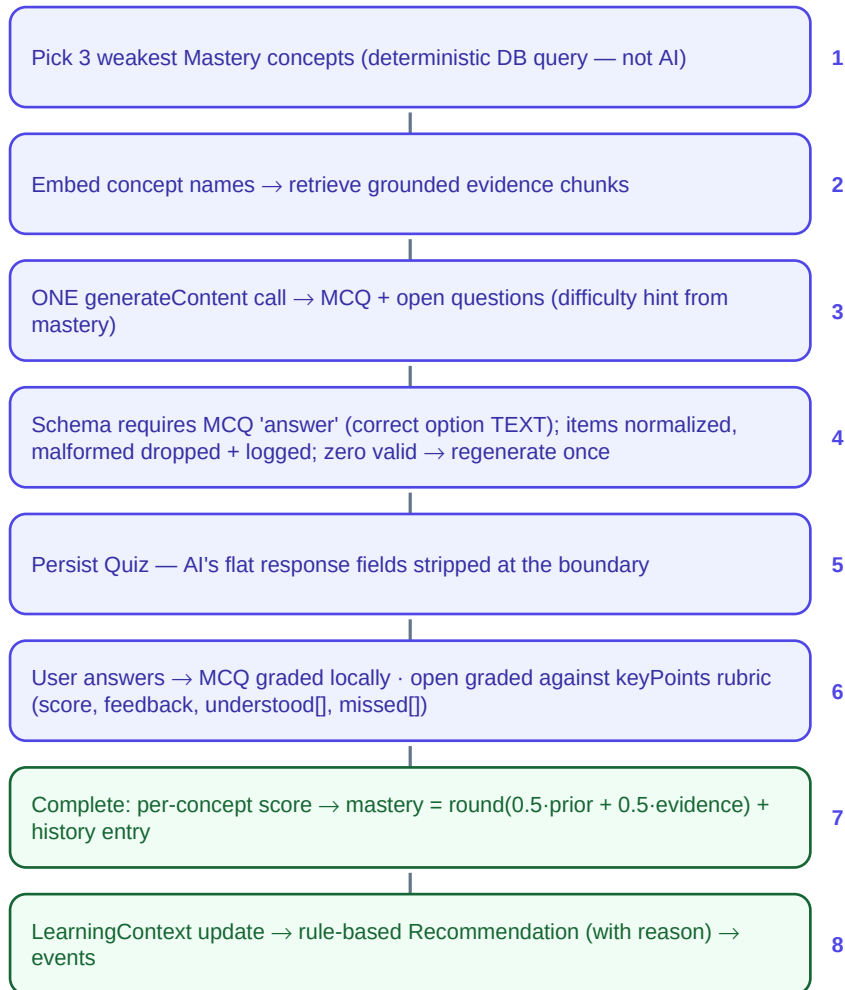
Collection	Purpose	Key design points
users	identity, role user/admin	unique email index; bcrypt hash only
spaces	broad learning area	owner indexed — every query filters on it
projects	learning workspace	requires goal (steers AI); space + owner indexed
materials	uploaded PDF + pipeline state	status enum queued/processing/ready/failed; raw bytes stored with select:false so retries are self-contained
chunks	searchable document slice	page + position for citations; embedding[768] for retrieval; project-scoped
concepts	key concepts per project	unique (project, name) index → idempotent upserts
mastery	0–100 estimate per concept	history[] of {level, at} powers growth trends
quizzes	quiz + embedded questions	correctIndex never serialized while active; per-question answer {score, feedback, understood, missed}
messages	tutor conversation	citations[] {material, materialName, page, snippet}; insufficientEvidence flag
recommendations	next-action suggestions	reason keeps them explainable; open/dismissed lifecycle
activityevents	append-only learning log	statics.log() fire-and-forget; feeds analytics + admin
aicalls	one row per AI provider call	feature, model, latencyMs, tokens, success, error
jobs	durable background queue	status, attempts/maxAttempts, availableAt for backoff

Deleting a Project cascades across all project-scoped collections (deleteProjectData) — nothing outlives its owner's intent.

4 · Request flow — grounded tutor turn

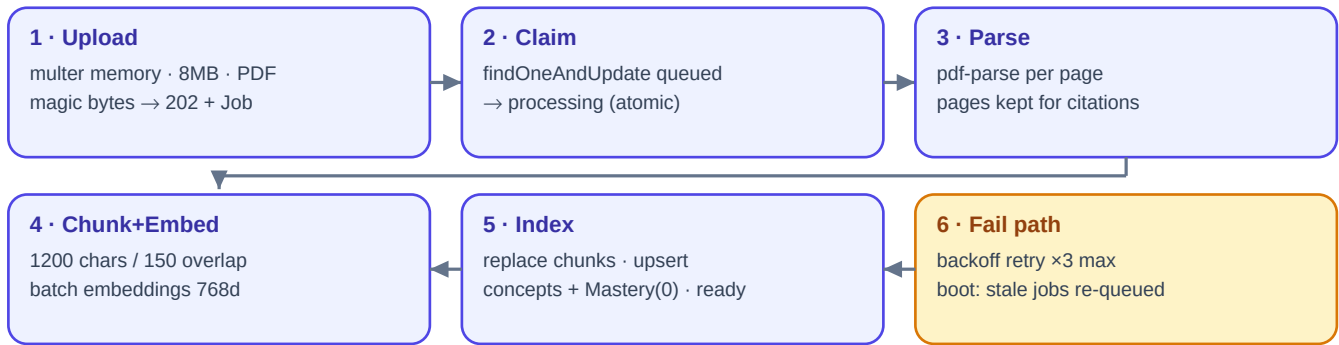


5 · Request flow — adaptive quiz loop



correctIndex derivation: exact text match → letter ('B') → integer fallback → bounds check; values-over-indexes because models are unreliable at counting.

6 · Request flow — background material pipeline



Upload returns **202** immediately — the UI polls material status every 3s while queued/processing. The worker lives in the same Node process (a separate service is listed under future work) and claims jobs atomically via `findOneAndUpdate`, so two ticks can never take the same job. Failure at any stage marks the job for retry with exponential backoff; after 3 attempts the job and material are marked **failed** with the error preserved for the admin dashboard (with a requeue action). Stale *processing* jobs found at boot are re-queued — crash recovery.

AI layer & hardening

All provider access flows through one gateway (`ai/gemini.js`): every call is tracked to `aicalls` (feature, model, latencyMs, tokens, success), transient 429/503 failures are retried with exponential backoff, and outputs are schema-constrained then re-validated and normalized before persistence. Model names are env config — a mid-build provider deprecation became a config change, not a code change.

Defense-in-depth on model output (each layer caught a real bug):

1. **response schema with required fields** — e.g. MCQ 'answer'; optional fields are exactly what stressed models omit
2. **parse guard** — non-array JSON rejected
3. **per-item normalization** — type coercion, defaults, malformed items dropped with the raw item logged
4. **values-over-indexes** — model returns the correct option's TEXT; code derives the index (exact → letter → int)
5. **content-level retry** — a response normalizing to zero valid questions triggers one regeneration
6. **friendly degradation** — sustained provider overload surfaces as a clean 503, never a raw 500

At the persistence boundary, AI field names are consumed/stripped before `Model.create` — a flat answer string once collided with the embedded answer subdoc (`ObjectExpectedError`), caught by the test suite.

7 · Key architecture decisions & trade-offs

Decision	Why (in plain words)	Trade-off / upgrade path
One language everywhere (MERN)	JavaScript runs the server, the browser and the AI calls — the fastest way to build alone in 3 days	Python has more ready-made AI testing tools
A jobs table instead of a queue service (no Redis)	retries, safe job claiming and crash recovery using just the database — nothing extra to run or pay for	all in one server process; at scale, move jobs to a separate worker service
Embeddings stored on chunks, similarity computed in our own code	no extra vector database to learn or pay for; a project has only a few hundred chunks	a vector search service would be faster beyond a few thousand chunks
PDF bytes kept inside MongoDB	a failed job can retry by itself because the file is always available; free hosts wipe saved files on restart	large files would move to object storage (S3) in production
Login token checked against the DB on every request	login stays stateless AND deleting a user blocks their token immediately	one small database lookup per request
Ownership written into every query	a new endpoint cannot forget isolation by accident; a stranger sees 'not found' — they cannot even tell the project exists	—
All AI calls through one gateway file	recording, retries and output checks live in one place — easy to find and fix	all provider code sits in one file, on purpose
Model names from environment variables	when a model is retired we change a setting, not the code — this actually happened mid-build	—
Recommendations from fixed rules, not AI	always explainable ('your weakest concept'), free, and predictable	AI-written suggestions could feel more personal — later
One AI call builds the whole quiz	fits the free limit of 20 generations per day	per-question calls when the budget allows

8 · Security

- **Passwords:** hashed with bcrypt before saving — the real password is never stored. A failed login never says which part was wrong, so nobody can discover which emails are registered.
- **Who can see what:** every request carries a login token; admin pages need a second check. Every database query also checks the owner — a stranger gets 'not found', never 'forbidden'.
- **Uploads:** max 8MB, PDF only — we check the file's real bytes, not the file type the browser claims. Updates only accept known fields.
- **AI safety:** uploaded documents and user messages are treated as data, never as instructions the AI must obey. AI replies are checked against a fixed structure before anything is saved.
- **Secrets:** API keys and the database URL live only in environment variables — never in the code or the repository. A `.env.example` documents the needed keys without real values.

9 · Watching the system (observability & reliability)

- **Every AI call is recorded:** which model, how long it took, how many tokens, success or failure. This answers 'why was this answer slow, expensive, or broken' — visible in the admin dashboard.
- **Background jobs remember their failures:** attempts and errors are stored, and failed jobs appear in the admin dashboard with a requeue button.
- **Every learning action is logged** (upload, question, quiz) — this feeds the analytics pages and the admin view.
- **One central error handler** turns known problems into clear answers: file too large → 400, duplicate email → 409, invalid id → 400, busy AI provider → a friendly 'try again' 503. Anything unexpected is logged and returns a plain 500.
- `/api/health` confirms the service and database are alive (Render checks it).

10 · Testing

15 tests in 6 suites, and they run fully offline — no internet, no API key, no daily quota. A fake in-memory database replaces MongoDB, and the AI is replaced by simple stand-ins plugged in at clearly marked seams.

What the tests check, in plain words:

- sign-up validation, duplicate emails, and that failed logins give nothing away
- a stranger opening another user's project gets 'not found'; logged-out users are blocked everywhere
- text splitting keeps page numbers and the right overlap between chunks
- similarity ranking puts the best-matching chunk first
- the whole upload → process pipeline runs against a real 2-page PDF and ends in the 'ready' state with correct pages
- the quiz loop checks the exact mastery math (30 → 15), blocks answering twice, and proves the correct answer never leaks to the browser before submission

Two tests exist only because of real bugs found during the build — the suite now pins those bugs down so they cannot come back.

11 · Deployment topology

Component	Platform	Notes
Frontend	Vercel (static, Vite build)	vercel.json rewrites all routes to index.html (SPA deep links); VITE_API_URL baked at build time
API	Render web service	npm start; /api/health as health check; free tier sleeps ~15 min (cold start ~50s)
Database	MongoDB Atlas M0	Render outbound IPs allowed; Mongoose connection at boot

Environment (all config, no literals in code):

Variable	Where	Purpose
MONGODB_URI	Render · local .env	Atlas connection string
JWT_SECRET / JWT_EXPIRES_IN	Render · local .env	token signing + lifetime
GEMINI_API_KEY	Render · local .env	provider credential (backend only — never in the browser bundle)
GEMINI_TEXT_MODEL / GEMINI_EMBEDDING_MODEL	Render · local .env	model IDs are config, not code
TUTOR_MIN_SIMILARITY	Render · local .env	refusal-gate threshold (0.3)
ADMIN_EMAIL	Render · local .env	this email registers as role=admin
CORS_ORIGINS	Render · local .env	comma-separated allowed origins
JOB_RETRY_DELAY_MS	Render · local .env	pipeline retry backoff base
VITE_API_URL	Vercel	deployed API base URL for the SPA

12 · Known limitations

- Free-tier AI quota (20 generations/day/model) and transient overload windows — handled (backoff, friendly 503, content retry), not eliminated
- Retrieval is chunk-level cosine over $\leq \sim 1k$ chunks — no reranking or cross-encoder
- Refusal threshold is a pragmatic constant (cosine ≥ 0.3), not per-project calibrated
- Scanned/image PDFs without a text layer are not OCR'd yet
- Single-process worker; vertical scale bounded by one Render instance
- JWT in localStorage (httpOnly cookie + refresh listed under future work)

13 · Future work (with more time)

Next features — drawn from the PRD's own 'should have' list:

- Streaming tutor — show the answer word by word instead of after a wait
- Richer document understanding — tables, diagrams and scanned pages (OCR)
- Improved analytics — filters, comparisons and longer trends
- Persistent tutor continuity — long-term memory of the conversation, not just the last few messages
- Background learning insights — automatic summaries of progress and weak areas
- Caching — remember answers and retrievals for repeated questions, cutting latency and quota use
- Deeper AI tracing — follow one request across AI calls, jobs and messages end to end
- Provider abstraction — add or swap AI models per feature behind the same gateway
- Automated regression evaluation — re-run a fixed set of golden questions on every prompt change
- Stronger workflow retries — dead-letter handling and smarter backoff for stuck jobs

Engineering upgrades beyond the PRD:

- Move the background worker into its own service (or a managed queue)
- Atlas Vector Search for faster retrieval at larger scale
- Login cookies JavaScript cannot read (httpOnly), refresh tokens, per-user rate limits
- Object storage (S3/GridFS) for PDF bytes instead of the database

Full incident history and evaluation approach: docs/ai-usage.md · setup and quickstart: README.md